

Real-time Data Processing Pipeline

TidaLine Maritime Logistics



Data Collection



Real-time Processing



Analytics

July 2025

Maritime Logistics Data Pipeline Implementation

🎯 Project Objectives

- ✓ Implement real-time seismic events monitoring for maritime risk assessment
- ✓ Develop batch processing pipeline for vessel and port data collection
- ✓ Create integrated data platform for maritime logistics operations

🧩 Implemented Components



Real-time Pipeline

PostgreSQL CDC with Debezium
Kafka message streaming
PySpark streaming processing
Snowflake data warehouse



Batch Pipeline

Vessel data web scraping
Port data API collection
HDFS data lake storage
PySpark batch processing



Infrastructure

Docker containerization
Automated scheduling
Data transformation framework
Bronze/Silver/Gold architecture



Data Sources



Seismic Events

Real-time PostgreSQL CDC



Vessel Data

VesselFinder web scraping

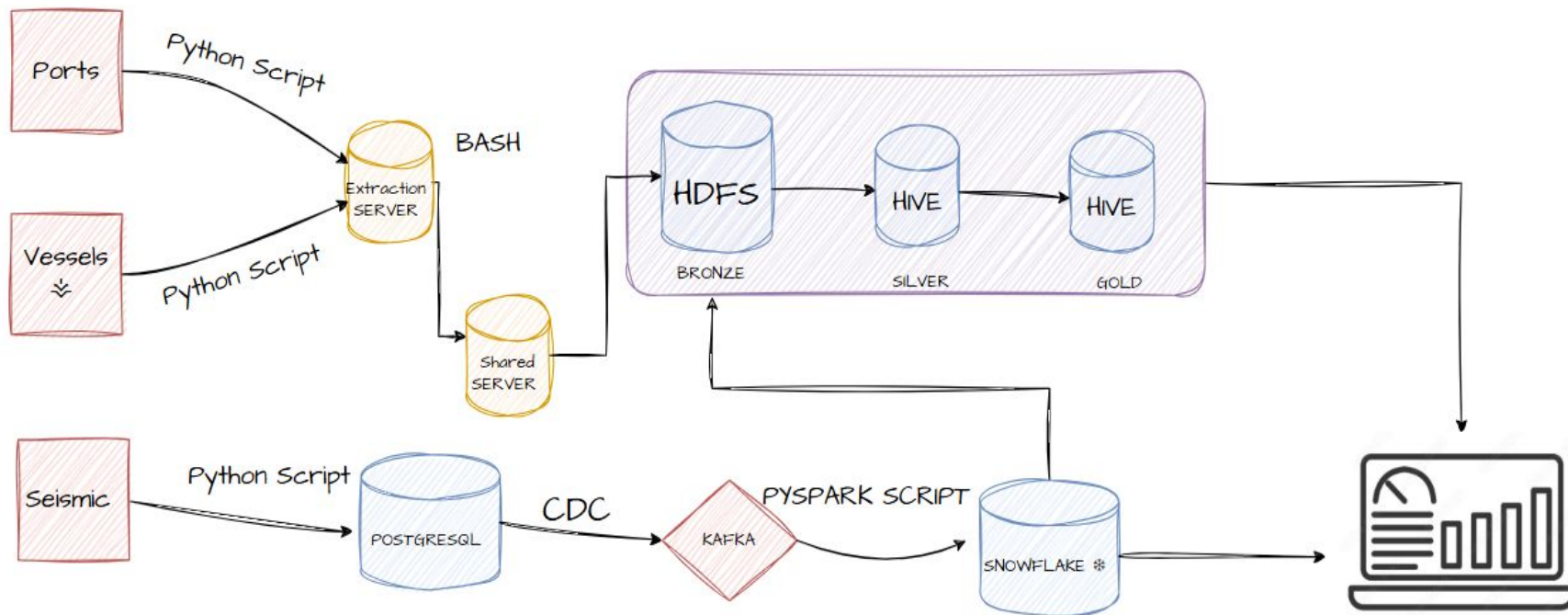


Port Data

NGA API downloads

Architecture Overview

Implemented Data Processing Pipeline



Architecture Overview

Implemented Data Processing Pipeline

Real-time Pipeline

PostgreSQL CDC with Debezium captures seismic event changes, streams through Kafka, and processes with PySpark streaming for real-time analytics in Snowflake.

Batch Pipeline

Web scraping collects vessel data, while API downloads gather port information. Data is stored in CSV files, processed through HDFS and PySpark batch jobs into Hive data warehouse.

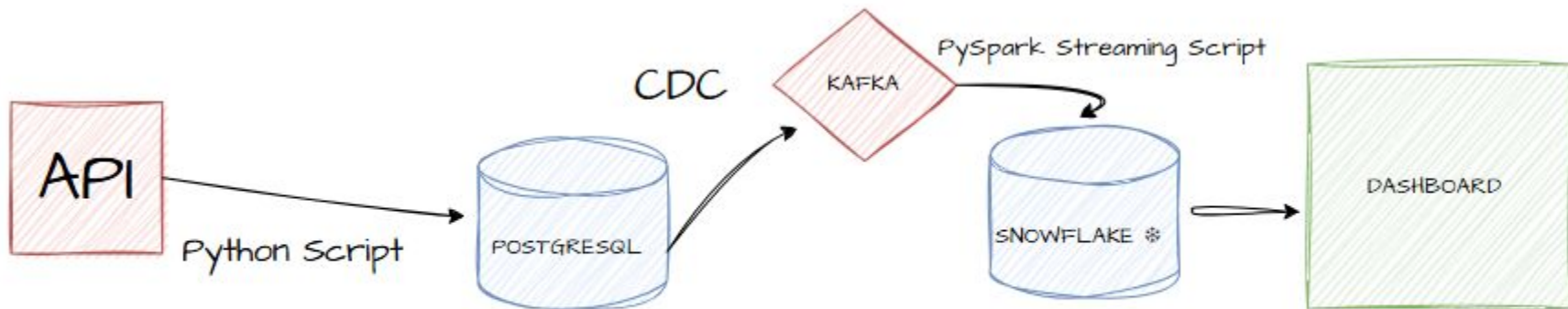
Data Integration

Both real-time and batch pipelines converge in HIVE data warehouse, enabling integrated analytics across seismic events, vessel positions, and port information.

Real-time Processing Pipeline

Seismic Events Monitoring System

⚡ Real-time Data Flow



Real-time Processing Pipeline

Seismic Events Monitoring System

➦ Change Data Capture Configuration

```
# seismic_connector.json
{
  "name": "seismic-events-connector",
  "config": {
    "connector.class":
"io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "postgres",
    "database.port": "5432",
    "database.user": "postgres",
    "database.password": "postgres",
    "database.dbname": "maritime_logistics",
    "database.server.name": "maritime_logistics_server",
    "plugin.name": "pgoutput",
    "slot.name": "seismic_events_slot",
    "publication.autocreate.mode": "filtered",
    "table.include.list": "public.seismic_events",
    "topic.prefix": "maritime_logistics_server"
  }
}
```

We set up a Debezium connector to track real-time changes in the [seismic_events](#) table from our PostgreSQL database. Whenever new events are added or existing ones are updated or deleted, this connector picks them up instantly.

It connects to the [maritime_logistics](#) database, watches only the [seismic_events](#) table, and streams the changes into Kafka topics. This lets us process seismic data in real time with tools like PySpark streaming, so we're always working with the latest information.

Real-time Processing Pipeline

Seismic Events Monitoring System

</> Streaming Implementation

```
# streaming_2_snowflake.py

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("Seismic Events Streaming") \
    .getOrCreate()

# Create streaming DataFrame from Kafka
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe",
"maritime_logistics_server.public.seismic_events") \
    .load()

# Write to Snowflake
query = df.writeStream \
    .format("snowflake") \
    .option("sfDatabase", "SEISMIC_DB") \
    .option("sfSchema", "BRONZE") \
    .option("dbtable", "EARTHQUAKE_EVENTS3") \
    .option("checkpointLocation", "/tmp/checkpoint") \
    .start()
```

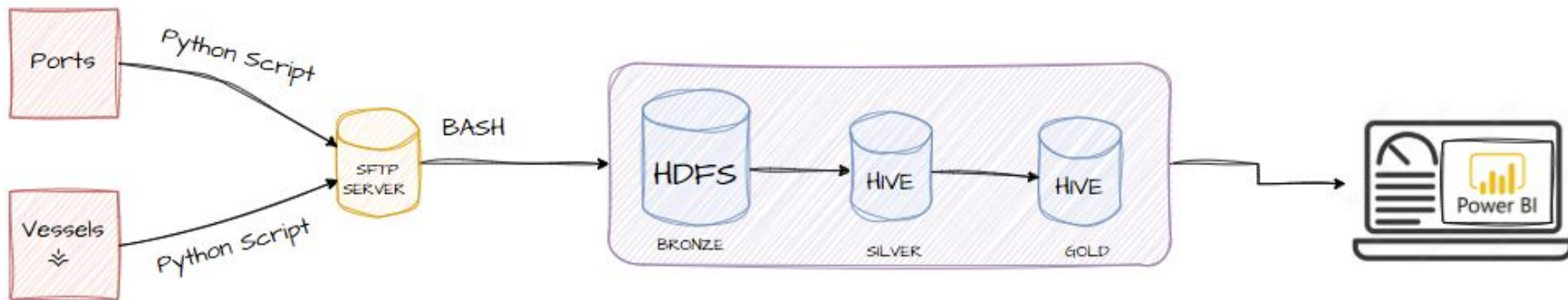
This script sets up a streaming job using PySpark to handle real-time seismic data. It starts by connecting to Kafka, where it listens to the [seismic_events](#) topic for any new or updated data coming from our PostgreSQL database (via Debezium).

As the data streams in, Spark continuously reads it and writes it directly into Snowflake, specifically into the [EARTHQUAKE_EVENTS3](#) table under the [BRONZE](#) schema. We also use a checkpoint **location** to keep track of what's already been processed, so the stream can resume smoothly if it restarts.

Batch Processing Pipeline

Vessel & Port Data Collection and Processing

Data Collection overflow



Batch Processing Pipeline

Vessel & Port Data Collection and Processing

Vessel Data Web Scrapping

```
class VesselScraper(Scraper):
    """
    A class to scrape vessel information from a given URL.
    """

    def __init__(self, base_url: str, endpoint: str = ""):
        super().__init__(base_url, endpoint)

    def get_page_soup(self, vessel_link) -> BeautifulSoup:
        """
        Get the BeautifulSoup object for the given URL.
        Args:
            url (str): The URL to scrape.
        Returns:
            BeautifulSoup: The BeautifulSoup object containing the page
            content.
        """
        url = f"{self.base_url}/{vessel_link}"
        response = requests.get(url, headers=self.header)
        if response.status_code != 200:
            raise Exception(f"Failed to retrieve page: {response.status_code}")
        return BeautifulSoup(response.text, 'html.parser')
```

We use a Python web scraper ([VesselScraper](#)) to collect vessel data from online maritime tracking websites that don't offer APIs. It builds the URL for each vessel, sends a request to get the webpage, and then uses BeautifulSoup to extract useful details like vessel name, type, and status. This data is later stored in the bronze layer and helps enrich the overall maritime dataset.

Batch Processing Pipeline

Vessel & Port Data Collection and Processing



Port Data API Collection

```
Base_url = os.getenv("Base_url", "https://msi.nga.mil")
Download_endpoint = os.getenv("download_endpoint",
"/api/publications/download?type=view&key=16920959/SFH00000/UpdatedPub150.csv")
Data_dir = os.getenv("data_dir", "/extraction/data/ports")
File_prefix = os.getenv("file_prefix", "WPI")
Log_file = os.getenv("Log_file", "/extraction/logs/download_ports_api.log")

def download_file_from_url():

    os.makedirs(Data_dir, exist_ok=True)
    now = datetime.datetime.now()
    file_name = f'{File_prefix}_{now.strftime("%Y-%m")}.csv'
    file_path = os.path.join(Data_dir, file_name)
    download_url = urljoin(Base_url, Download_endpoint)
    try:
        response = requests.get(download_url)
        response.raise_for_status()
        with open(file_path, 'wb') as file:
            file.write(response.content) the file.
        logging.info(f"File downloaded successfully: {file_path}")
    except requests.exceptions.RequestException as e:
        logging.error(f"Error downloading file: {e}")
    except Exception as e:
        logging.error(f"Unexpected error: {e}")
```

We use this Python script to **automatically download updated port data** (like shipping waypoints) from a public government source. It builds a dynamic URL using environment variables, fetches the CSV file, and saves it locally with a date-stamped name for versioning. Logging is also set up to track any issues during the download process. This ensures we always have the latest static port information feeding into our **bronze layer**, supporting our broader maritime analysis.

Batch Processing Pipeline

Vessel & Port Data Collection and Processing

HDFS Processing Pipeline

1 Bronze Layer: Raw Data Ingestion

it's where we collect everything from the outside world without judging or cleaning it yet.

- We bring in raw data from different sources — scraped vessel info, downloaded port CSVs, API calls, etc.
- The goal here is just to collect and store it safely, exactly as it arrives.
-
- Nothing is transformed yet — it's the untouched truth from the source.

2 Silver Layer: Data Transformation

this is where data gets cleaned, structured, and made usable.

- We fix missing values, clean formats (like timestamps and IDs), and maybe enrich it with extra context (like matching vessels to port locations).
- This is the layer that makes raw data **understandable and consistent**.
- It's where we prepare data to be used for serious analysis.

3 Gold Layer: Analytics-Ready Data

This is the polished, final version of the data that the business team or dashboard wants to see.

- It includes views like “How many vessels are in port?”, “What’s the average wait time?”, etc.
- Data is **filtered**, aggregated, and optimized to answer real business questions.
- This is what connects our backend data pipeline to tools like **Power BI**.

Data Integration & Analytics

Combining Real-time & Batch Processing for Maritime Intelligence

HIVE Integration

Seismic Events Table Schema

```
create or replace TABLE
SEISMIC_DB.BRONZE.EARTHQUAKE_EVENTS4 (
  PK VARCHAR(16777216),
  UNID VARCHAR(16777216),
  SOURCE_ID VARCHAR(16777216),
  SOURCE_CATALOG VARCHAR(16777216),
  LASTUPDATE NUMBER(38,0),
  TIME NUMBER(38,0),
  FLYNN_REGION VARCHAR(16777216),
  LAT FLOAT,
  LON FLOAT,
  DEPTH FLOAT,
  EVTYPE VARCHAR(16777216),
  AUTH VARCHAR(16777216),
  MAG FLOAT,
  MAGTYPE VARCHAR(16777216),
  ACTION VARCHAR(16777216),
  RECEIVED_AT NUMBER(38,0),
  SOURCE_DB VARCHAR(16777216),
  SOURCE_TABLE VARCHAR(16777216),
  OPERATION VARCHAR(16777216),
  LOAD_TIMESTAMP NUMBER(38,0),
  BPK VARCHAR(16777216)
);
```

Integrated Data Samples

Vessel Data Sample (Silver Layer)

name	type	year_built	last_port	destination
ATHINEA	Tanker	2006	As Suways (Suez)	RAS GHARIB
ETC RAMSIS	Tanker	2003	As Suways (Suez)	Wadi Feiran

Port Data Sample (Silver Layer)

Main_Port_Name	Country_Code	Harbor_Type	Latitude	Longitude
Maurer	United States	River (Natural)	40.533333	-74.25
Iharana	Madagascar	Coastal (Natural)	-13.35	50.0

Snowflake View for Analytics

```
-- View created for batch processing
CREATE OR REPLACE VIEW EARTHQUAKE_EVENTS3_VIEW AS
SELECT
  UNID,
  SOURCE_ID,
  TIMESTAMP_NTZ_FROM_PARTS(1970, 1, 1, 0, 0, 0) +
    TIME::INTEGER / 1000 AS EVENT_TIME,
  LAT, LON, DEPTH, MAG,
  TIMESTAMP_NTZ_FROM_PARTS(1970, 1, 1, 0, 0, 0) +
    RECEIVED_AT::INTEGER / 1000 AS RECEIVED_TIME
FROM SEISMIC_DB.BRONZE.EARTHQUAKE_EVENTS3;
```

Conclusion

Maritime Logistics Data Pipeline Implementation

Key Achievements

Real-time Seismic Monitoring

Successfully implemented PostgreSQL CDC with Debezium to capture seismic events in real-time.

Comprehensive Vessel Data Collection

Developed web scraping system to collect vessel information from VesselFinder.

Automated Port Data Collection

Created scheduled API download system to retrieve port data from NGA.

Integrated Data Platform

Established unified data platform in HIVE combining real-time and batch data.

Technical Implementation Summary

Real-time Pipeline

PostgreSQL CDC with Debezium
Kafka message streaming
PySpark structured streaming

Batch Pipeline

Web scraping for vessel data
API downloads for port data
HDFS data lake storage

Data Architecture

Bronze layer raw data storage
Silver layer transformed data
Gold HIVE unified warehouse

Business Value

Maritime Risk Assessment

Real-time correlation of seismic events with vessel positions and port

Operational Intelligence

Comprehensive vessel tracking and port information combined with